

Jenis Skema Pengulangan

1. Iterate-stop
2. Repeat-until
3. While-do
4. Repeat N-times
5. Traversal

Skema Pemrosesan Sekuensial

1. Dengan **MARK**

- a. Tanpa penanganan kasus kosong secara khusus
while not (EOP) do
 Process_Current_Elmt
 Next_Elmt
 { EOP }
-> maksudnya, saat memenuhi EOP, tidak ada pesan yang ditujukan ke pengguna bahwa mereka telah mencapai EOP
- b. Dengan penanganan kasus kosong
If (EOP) then
 Proses_Kasus_Kosong
else
 repeat
 ...
 until (EOP)

2. Tanpa **MARK** (*pasti tidak ada kasus kosong*)

Studi Kasus 1

Program Penjumlahan

{Spesifikasi program: Membaca bilangan positif N, lalu menuliskan 1, 2, 3... N dan menjumlahkan bilangan 1 sampai N, serta menuliskan hasil penjumlahannya}

PROGRAM

- a. Iterate-stop
 i <- 1
 jumlah <- 0
 iterate
 output(i)
 jumlah <- jumlah + i
 stop (i = N)
 i <- i+1
 output(jumlah)

Penjelasan:

- Tanpa Mark, karena nilai terakhir masuk ke dalam data

- Program benar dengan tambahan spesifikasi $N \geq 1$. Kalau ga ditambah spesifikasi, program kurang optimal

b. Repeat-until

```



```

Penjelasan:

- Tanpa MARK
- Program benar dengan tambahan spesifikasi $N \geq 1$ (sama kaya iterate-stop)
- Nilai i terakhir di i+1

c. Repeat N times

```



```

Penjelasan:

- Kurang optimal karena fungsi skema repeat n times bukan untuk ini

d. While-do

```



```

Penjelasan:

- Tanpa MARK
- Jika $N < 1$, program tetap optimal, terjadi kasus kosong
- Nilai i terakhir sama dengan N

e. Traversal

```



```

```
jumlah <- jumlah+i
  output(i)
output(jumlah)
```

Penjelasan:

- Tanpa MARK
- Jika $N < 1$, program tetap optimal, terjadi kasus kosong. SALAH. N harus ≥ 1 . Hal ini karena cara kerja traversal otomatis oleh sistem, bukan ditetapkan syaratnya oleh pemrogram, sehingga pemrogram bisa mengalami kebingungan dengan hasil yang dikeluarkan saat $N < 1$. Beda dengan while yang pemrogram secara sadar menuliskan syarat keberjalanan program
- Nilai i terakhir sama dengan N, tetapi nilai i lokal di dalam traversal

Studi Kasus 2

Program Penjumlahan2

{Spesifikasi program: Membaca nilai integer yang dibaca dan menjumlahkan nilainya.
Pemasukan nilai diakhiri dengan 9999}

PROGRAM

a. Iterate-stop

```
jumlah <- 0
iterate
  input(N)
stop (N = 9999)
  jumlah <- jumlah + N
output(jumlah)
```

Penjelasan:

- Efektif
- Dengan MARK tanpa penanganan kasus kosong
- bisa dibikin ada penanganan kasus kosong dengan nambahin if kaya di repeat until di bawah

b. Repeat until

```
jumlah <- 0
input(N)
if(N != 9999) then
  repeat
    output(N)
    jumlah <- jumlah + N
    input(N)
  until (N = 9999)
else
  output("Kasus kosong bray")
```

Penjelasan:

- Efektif
 - Dengan Mark dengan penanganan kasus kosong
- c. While-do

```

jumlah <- 0
output(N)
    jumlah <- jumlah + N
    input(N)
output(jumlah)

```

Penjelasan:

- Tidak ada penanganan kasus kosong, harusnya bisa jg kalo ditambah if kaya di repeat until
- d. Traversal (tidak bisa)
- e. Repeat N times (tidak bisa)

Studi kasus 3

Program Penjumlahan3

{Spesifikasi program: Membaca nilai integer yang dibaca, menjumlahkan nilainya, dan mencacah banyak integer yang dibaca. Pemasukan nilai diakhiri dengan 9999}

PROGRAM

- a. Iterate-stop
- ```

Total <- 0
Jumlah <- 0
iterate
 input(N)
 stop (N = 9999)
 Total <- Total + N
 Jumlah <- Jumlah + 1
output(Total, Jumlah)

```

Penjelasan:

- Tdk ada penanganan kasus kosong, tp sama kek tad8
- b. Repeat until
- ```

Total <- 0
Jumlah <- 0
then
    output("Kasus kosong")
else
    repeat
        Jumlah <- Jumlah+1
        Total <- Total+N

```

```
        input(N)
    until (N = 9999)
    output(Total, Jumlah)
```

Penjelasan:

- Ada penanganan kasus kosong
- c. While do
sama kek tadi
- d. Repeat N times
- e. Traversal

Latihan 2

Program NilaiUTS

{Spesifikasi program: Membaca sejumlah nilai UTS di suatu kelas. Nilai yang valid adalah 0...100. Pembacaan berhenti saat nilai yang dimasukkan di luar range. Di akhir, ditampilkan rata-rata nilai UTS. Jika tidak ada nilai UTS yang dimasukkan, tuliskan "Tidak ada data"}

PROGRAM

- a. Iterate Stop (tidak ada penanganan kasus kosong), tp bisa kaya yg tadi jg harusnya
- b. Repeat until -> pake ini

```
Total <- 0
Jumlah <- 0
input(Nilai)
if(Nilai > 100 or Nilai < 0) then
    output("Tidak ada data")
else
    repeat
        Jumlah <- Jumlah+1
        Total <- Total+Nilai
        input(Nilai)
    until (Nilai > 100 or Nilai < 0)
    output(Total/Jumlah)
```

- c. While do

```
Total <- 0
Jumlah <- 0
input(Nilai)
if(Nilai > 100 or Nilai < 0) then
    output("Tidak ada data")
else
    while(Nilai <= 100 and Nilai > 0) do
        Jumlah <- Jumlah + 1
        Total <- Total + Nilai
        input(Nilai)
```

output(Total/Jumlah)

d. Repeat n times

e. Traversal

Latihan 3

Program Validasi2Jam

{Spesifikasi program: melakukan validasi input pada formatting jam}

KAMUS

type jam: < HH : integer[0...23];
MM : integer[0...59];
SS : integer[0...59] >

J : jam

h, m, s : integer

PROGRAM

```
input(h, m, s)
while(h > 23 or h < 0 or m > 59 or m < 0 or s > 59 or s < 0) then
    output("Tidak dapat membentuk jam")
    input(h, m, s)
J.HH <- h
J.MM <- m
J.SS <- s
```

Latihan 4

Program UTSUAS (ini belum efektif)

{Spesifikasi program: Membaca nilai UTS dan UAS, jdiakhisi jika nilai masukan UTS di luar range nilai, lalu menghitung dan mencetak rata-rata nilai akhir. Digunakan skema validasi 2 untuk nilai UAS}

PROGRAM

```
Jumlah <- 0
All <- 0
input(UTS)
if(UTS > 100 or UTS < 0) then
    output("Data kosong")
else
    repeat
        iterate
            output("Ulangi input nilai (0...100) !")
            output("Nilai UAS = ")
            input(UAS)
```

```
stop (UAS <= 100 and UAS < 0)
      output("Ulangi input nilai (0...100) !")
      output("Nilai UAS = ")
      input(UAS)
Jumlah <- Jumlah + 1
Total <- 0.4*UTS + 0.6*UAS
All <- Total
output("Nilai rata-rata dari pelajaran", Jumlah, " = ", Total)
input(UTS)
until (UTS = 9999)
```